



ВВЕДЕНИЕ В ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ: ОСНОВЫ HASKELL

Дисциплина Языки и системы программирования

История

Для начала, вернемся в далекие 20е годы 20 века, от появления первого компьютера нас отделяет 20-25 лет, но в мире существует ряд проблем, для которых такого рода устройства были бы очень полезными. Одна из таких проблем, проблема разрешимости, ответ на вопрос “да или нет?”, а точнее вопрос о возможности создания алгоритма, который мог бы на основании заданных аксиом дать ответ на то верно ли какое-то логическое высказывание.

Это довольно сложная математическая проблема, которая захватывала умы многих математиков того времени. Она заставила задуматься над механизмами решения проблемы в целом. Для решения необходимо было изобрести понятие вычислимости, понимая ее как процесс автоматизации решения проблемы используя небольшие, переиспользуемые части. Несколько человек работали над этой идеей параллельно и не смотря на то, что в итоге пришли к одному и тому же, использовали кардинально разные подходы. Один из которых является основой функционального программирования.

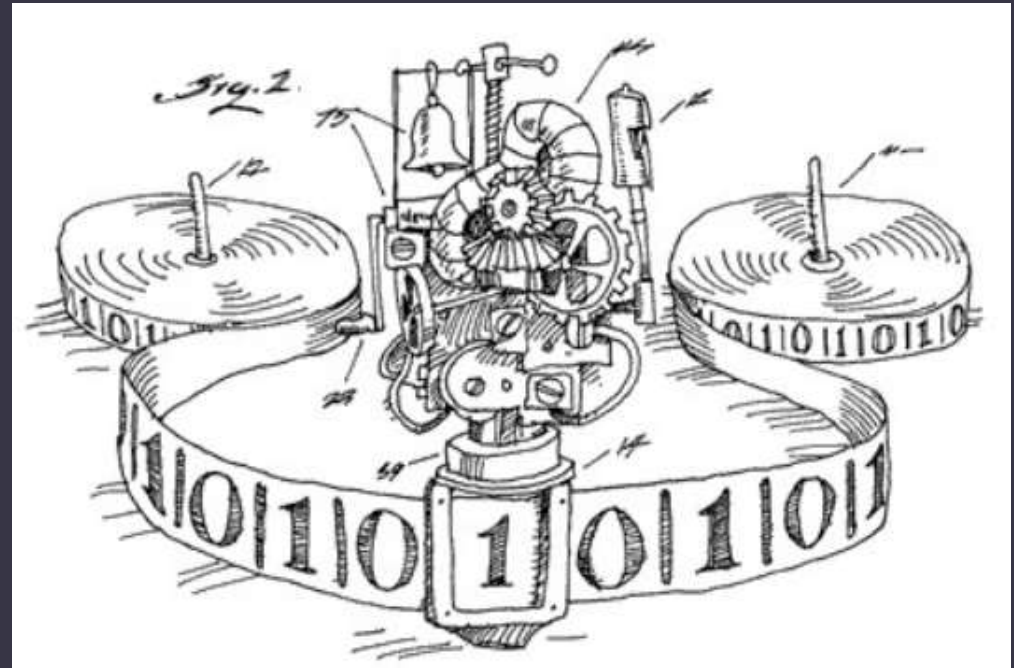
Решение проблемы вычислимости

Один из них, знаменитый британский ученый Алан Тьюринг, использовал максимально прагматичный и доступный подход (1936г). Он представлял себе процесс вычислимости механизированным. Действительно, его идея то, что мы сегодня называем “Машина Тьюринга”, несмотря на то, что она так и никогда не существовала. “Машина Тьюринга” больше ментальное упражнение, чем реальный механизм.



Машина Тьюринга. Механистический подход

Тьюринг предложил простой подход для решения проблемы вычислимости. Он предложил идею абстрактной машины, которая имеет специальное устройство считывания/записи, бесконечную ленту в обе стороны, разделенные на ячейки, 2 бобины с обоих концов машины для движения ленты и управляющее устройство с конечным числом состояний.



Управляющее устройство может двигать ленту влево и вправо, читать и записывать в ячейки символы некоторого конечного алфавита. Выделяется особый пустой символ, заполняющий все клетки ленты, кроме тех из них (конечного числа), на которых записаны входные данные. В управляющем устройстве содержится *таблица переходов*, которая представляет алгоритм, реализуемый данной Машиной Тьюринга.

Каждое правило из таблицы предписывает машине, в зависимости от текущего состояния и наблюдаемого в текущей клетке символа, записать в эту клетку новый символ, перейти в новое состояние и переместиться на одну клетку влево или вправо. Некоторые состояния Машины Тьюринга могут быть помечены как *терминальные*, и переход в любое из них означает конец работы, остановку алгоритма.

Что в целом можно описать как набор последовательных операций.

Лямбда-исчисление. Функциональный подход

С другой стороны, американский ученый [Алонзо Черч](#), разработал более математический подход (1936г), который является основной современных функциональных языков программирования - лямбда-исчисление.



Лямбда-исчисление представляло собой более элегантный математический подход, по сути это формальный аппарат, способный определить в своих терминах любую языковую конструкцию или алгоритм. В самом простом виде оно состоит из нескольких элементов - термов и функций и применения функций. Все функции являются анонимными, т.е. не имеют конкретного названия, также называются лямбда. Функции можно применять как к термам так и к другим функциям, т.е. функции можно передавать в виде аргумента в другие функции.

$\text{---} : \text{---} \mid \text{---} : \text{---} : \text{---}$

переменная $\mid x \mid$

лямбда $\mid \lambda x.t$, где x — аргумент функции, t — её тело $\mid$

применение $\mid f x$, где f — функция, x — подставляемое в неё значение аргумента $\mid$

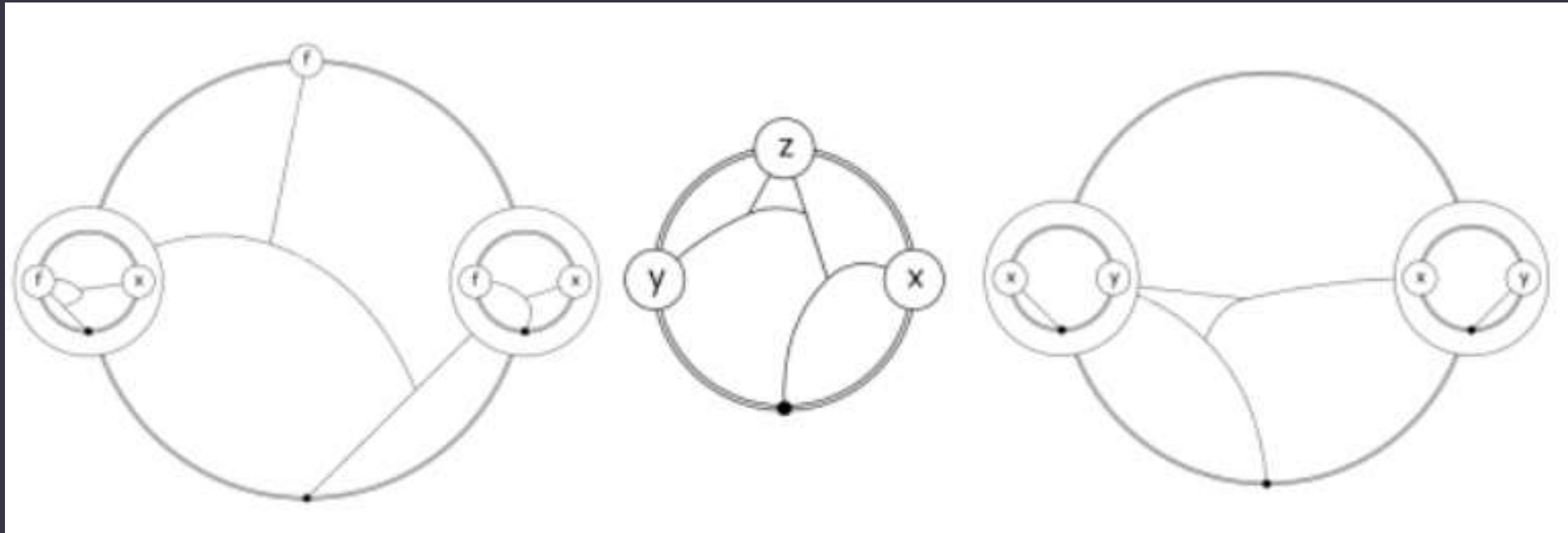
Любой алгоритм можно разбить на последовательность применения функций. Даже натуральные числа можно представить в качестве последовательности применения лямбда выражений, например:

```
0 = λf.λx.x  
1 = λf.λx.f x  
2 = λf.λx.f (f x)  
3 = λf.λx.f ( f (f x))  
...
```

Обратите внимание на конечную часть, каждое следующее число, это применение функции к предыдущему, но здесь последовательности в явном виде это применение, поэтому конечный алгоритм будет содержать не набор инструкций, а одну большую функцию, вычислив которую можно будет получить результат.

Эта идея стала основной для развития функциональных языков, зная ключевые особенности лямбда-исчисления, становятся более понятными подходы конкретных функциональных языков программирования.

Над этой теорией так же трудилось множество знаменитых ученых как Хаскелл Брукс Карри, Стивен Клини, Эмиль Прост, которые вложили существенный вклад в развитие данного направления.



Развитие функциональных языков

Состояние элементной базы периода второй мировой войны не позволяло организовать вычислитель на базе лямбда-исчисления, поэтому на вооружение была взята идея Машины Тьюринга, какой-то период времени существовало несколько видов используемых архитектур Гарвардская и архитектура фон Н്യюмана. Опять таки в силу технических и бизнес причин, победу одержала архитектуру фон Н്യюмана. Практически любой современный процессор, конечно во много раз более сложнее изначальных вычислителей, однако базируются на первоначальных принципах архитектуры фон Н്യюмана. Ключевым отличием Гарвардской архитектуры было разделение шины данных и шины инструкций. В такой архитектуре процессор может читать инструкции и выполнять доступ к памяти данных одновременно, без использования кэш-памяти. Компьютер при определенной сложности схемы быстрее, чем компьютер с архитектурой фон Неймана, поскольку шины инструкций и данных расположены на разных, не связанных между собой физически, каналах. Эта архитектура используется в некоторых современных чипах Atmel, AVR и так же показательна, что не всегда лучший подход побеждает при выборе технологий.

Lisp

В 1958 году в недрах MIT появился близкий к функциональному язык программирования Lisp, который был вторым высокоуровневым языком программирования, созданный всего на год позже Fortran. Язык был разработан Джоном Маккарти. Язык был динамический, предполагал, что код есть данные, позволявшие различные манипуляции с исходным кодом на лету, как будто это часть данных, которые можно изменять, а не жестко зафиксированный код в виде скомпилированной программы.

Lisp машины

Изначально реализация языка была создана для типичного компьютера того времени IBM 704, однако дальнейшее десятилетие разработок привело к созданию уникальных для того времени LISP машин, которые представляли собой аппаратную реализацию архитектуры для вычисления выражений лямбда-исчислений. Помимо этого обладали рядом преимуществ, которые для того времени были уникальными. Такие как:

- автоматическая сборка мусора на уровне ОС
- сохранение состояния без питания

Первое позволяло избавиться от рутинной работы с памятью и множеством проблем, с которыми сталкиваются разработчики на C, C++. Второе, позволяло отключить питание компьютера, а затем включить как будто ничего не произошло, полное сохранение состояние машины.

Одной из ключевых концепций языка является REPL - Read Eval Print Loop, механизм, который затем перекочевал во многие другие языки программирования и представляет из себя очень эффективный механизм для повышения продуктивности разработчика. Это интерактивная среда, которая позволяет вводить выражения и моментально получать результат, представляя собой постоянную обратную связь. Данная среда может быть реализована как для интерпретируемых, так и для компилируемых языков программирования.

Развитие Lisp семейства поддерживало интерес к развитию функциональных языков, однако основной толчок к развитию дали работы различных ученых в 80х.

Особенности Функционального программирования

Разберемся в ключевых аспектах, которые характерны для большинства функциональных языков программирования и делает их интересными для изучения.

Функции основа всего

Так как большинство функциональных языков базируются на лямбда-исчислении они оперируют функциями. Функция это минимальная логическая частица, которой мы оперируем, мы можем *писать функции, передавать их в качестве аргументов, возвращать в качестве результата выполнения функции, создавать анонимные функции и применять к ним различные операции.*

Неизменчивость (*immutability*)

Способность возвращать каждый раз измененную структуру данных, а не оперировать одной и той же структурой. Изменение передаваемых структур данных может привести к печальным последствиям в виде доступа к участкам памяти, которых не существует. Такая способность позволяет избавиться от такого рода проблем, что в свою очередь ведет к колоссальному упрощению организации параллельного исполнения программ.

Чистота. Отсутствие побочных эффектов

Отсутствие побочных эффектов, позволяет быть уверенным, что функция всегда работает одинаково при одних и тех же входных данных и не зависит от состояний передаваемых объектов.

Сравнение ФП с другими ПОДХОДАМИ

В сравнении с процедурным программированием

Процедурное программирование представляет разработчику думать в рамках операций, которые необходимо выполнить процессору для того, чтоб достигнуть определенного результата. Во многом прикладную модель реализации практически невозможно выразить и она во многом будет зашумлена особенностями реализации. Конечно, в свое время это был скачок в развитии продуктивности как отдельно взятого программиста так и команд в целом, однако, современный разработчик должен понимать прикладную область и стараться выражаться ее терминологией, скрывая конкретные реализации. Функциональное программирование позволяет довольно близко имитировать прикладную область без существенного ущерба реализации.

В сравнении с объектно-ориентированным программированием

Само понятие ООП так же по разному понимается в различных языках программирования, наиболее приближенным к изначальным идеям считается [SmallTalk](#), которые умеет делать некоторые полезные вещи из мира параллельного программирования, но это отдельная тема.

Больше всего ассоциированные с ООП C++, Java, C# понимают ООП по своему, т.к. во-первых используют только часть изначальных идей и в том контексте как мыслит разработчики языка. Однако, позволяют намного ближе продвинуться к моделированию прикладной области в ее терминах, что сделало данный подход очень популярным. Проблема ООП в том, что он пытается оперировать состояниями объектов и их изменением, зачастую процесс изменения не отслеживается и может приводить к некорректной работе, о чистоте описанной выше говорить невозможно, неизменчивость достигается в разных языках с помощью специальных механизмов, которые направлены на блокирование доступа к объектам и ведет к большим сложностям при разработке.

Функциональное программирование позволяет избавиться от многих проблем благодаря своим изначальным подходам. Сконцентрировавшись на прикладной области.

Множество функциональных языков

Common Lisp

Один из большого семейства Lisp языков, который получил свой стандарт и имеет наибольшую распространенность в индустрии. На данный момент степень его использования значительно ниже, чем в начале 2000х, однако он содержит в себе все ключевые идеи, которые были заложены в сообществе Lisp и имеет мощную систему расширения для работы с объектами CLOS.



Clojure

Это функциональный язык на базе платформы JVM и представляет собой реинкарнацию идей Common Lisp на одной из самых распространенных программных платформ. Помимо классических преимуществ он имеет ряд уникальных свойств и технологий, которые характерны для него. На данный момент имеет довольно активное сообщество и может использовать как свой набор библиотек, так и библиотеки реализованные на Java, благодаря взаимодействию через JVM.



Erlang

Этот язык был разработан компаний Ericsson в начале 80х для организации обработки телефонных сигналов. В тот период времени зарождались системы типа SoftSwitch, которые позволили вывести массовую обработку вызовов на новый уровень. Со временем язык получил значительное развитие и сейчас используется во многих отраслях. Ключевой идеологией является - если что-то может сломаться пускай “Let it crash”.



OCaml

Это один из самых развиваемых языков семейства ML. Он не является полностью функциональным, но активно использует многие концепции. Обладает мощной системой автоматического вывода типов, что позволяет его использовать в критически важных проектах или частях программ.



F#

Большей частью похож на язык программирования OCaml, разрабатывается компанией Microsoft и базируется на платформе .Net что позволяет использовать уже спектр готовых библиотек и возможностей языка C# если необходимо. Так же обладает выводом типов, позволяя использовать в важных участках проекта.



Haskell

Другой язык программирования Haskell имеет все основные особенности функционального программирования, является чистым, ленивым языком, с развитой системой типов и самой мощным на данный момент механизмом вывода типов.

Сам язык имеет более 20 лет истории, активно развивается и поддерживается большим сообществом разработчиков по всему миру. Он вобрал в себя основные преимущества функциональной парадигмы и снабжен самой современной системой статической типизации и вывода типов.



Чистые функции

Это функция, которые при одних и те же входных данных возвращают одинаковый результат. Отсутствие оперирования состояниями, позволяют организовать такого рода функции.

Функции высшего порядка

Это функции, которые можно передавать в качестве аргументов или возвращать в качестве аргументов, выполнять каррирование и композицию.

Ленивость

Ленивость это возможность, которой обладают немногие языки. Это способность отложить вычисления до того момента когда они будут действительно необходимы.

Емкость

Как и все языки функционального программирования обладает набором абстракций, которые позволяют избежать множества довольно витиеватого кода и думать больше в концепциях прикладной области. Ключевыми элементами, обеспечивающими емкость:

- сопоставления-по-образцу(pattern-matching), который позволяет избавиться от if-then-else “лапши” в коде.
- абстракции над списками, которые позволяют избавиться от множества циклов
- хвостовая рекурсия и культура рекурсивных функций в целом.

Автоматический вывод типов

Это наверное самая важная часть языка, которая выделяет его в отдельную область и делает примечательным. Это уникальная система вывода типов, которая позволяет избегать практически всех известных ошибок компиляции. Позволяет не описывать типы и можно утверждать, что она не будет содержать ошибок. Выводить их из логики работы самой программы. Если программа составлена верно и скомпилирована, то можно утверждать, что она не будет содержать ошибок.

История Haskell

Над Haskell начали работу в 1988 году, комитетом ученых, который в течении года разработал концепции функционального языка, который будет обладать исключительно преимуществами, конечно многие моменты тяжело было продумать изначально, поэтому язык развивается до сих пор и активно поддерживается сообществом разработчиков по всему миру.

Haskell экосистема

Для того, чтоб начать работу с Haskell необходимо знать о том, как построена его экосистема. Экосистема включает в себя ряд программ, которые необходимы для эффективной работы.

GHC

GHC - (Glasgow Haskell Compiler) - это единственный на данный момент компилятор, активно развиваемый и используемый сообществом, он обеспечивает компиляцию согласно стандарту и имеет множество расширений, для обхода определенных ограничений языка в прикладном применении. Если вы хотите собрать ваши Haskell программы, он понадобится вам в обязательном порядке.

Cabal, Hackage, Haskell-Platform

Cabal - представляет собой программу, которая выполняет функции пакетного менеджера и сборщика программ, расшифровывается как Common Architecture for Building Applications and Libraries, т. е. «общая архитектура сборки приложений и библиотек».

Эта программа разрабатывается уже более 10 лет, изначально создана Исааком Джонсом, а сейчас активно развиваемая Йоханом Тиббе (Johan Tibbe) и десятком постоянных контрибьюторов. Последние несколько лет программа активно развивается, благодаря тесному взаимодействию с Haskell сообществом.

GHCI (REPL)

GHCI это среда для интерактивного взаимодействия с Haskell программами, которая позволяет вводить функции и мгновенно получать результат их выполнения, изменять их состав и без перекомпиляции всей программы. Эта система позволяет проводить эффективное моделирование и быстрое тестирование кода, благодаря отсутствию задержки в обратной связи. Это один из ключевых инструментов при разработке на Haskell.